GitHub project: https://github.com/kanishkzzz/Payshift

Live app: https://payshift-eight.vercel.app

Tracky Technical Design Document

Coaching institute fee tracking system and parent portal

Prepared	Source snapshot	Product status
8 November 2026	Local working tree	MVP in active development

Executive summary

Tracky connects fee schedules, parent access, payment allocation, receipts, dues reporting, refund review, reminder automation, and counsellor follow-up in a single web application. The implementation is a Next.js modular monolith backed by PostgreSQL. Financial calculations use integer paise, while risk flags are explainable rule-based signals that a person reviews.

The product is intentionally presented as a working MVP. The mock payment provider is suitable for a demo but does not move money; production SMS and WhatsApp delivery are not connected; Tally support is a CSV export; and the risk system does not call an LLM.

Assumptions and scope

Label	Assumption used in this design
A1	The submission text contains no separate product problem statement. This TDD treats the current Tracky HLD, source tree, and deployed landing page as the selected product scope.
A2	Seeded students, guardians, staff, fee schedules, and contact details are synthetic demo records. The reviewer deployment uses a disposable database.
A3	The fixed reviewer parent OTP is a demo convenience restricted to one seeded parent phone. It is not a production OTP delivery service.
A4	External integrations are considered active only when configured. No SMS/WhatsApp provider, connected LLM, live Tally connection, or real payment settlement is assumed.

Users and core jobs

User	Primary job
Parent or guardian	Sign in by phone OTP, see linked students and fee balances, pay an assigned instalment, find receipts, and set the language used for fee reminder messages.
Accounts staff or administrator	Review centre-scoped dues, reconcile unmatched payments, inspect refund cases, and export dated receipt data.
Counsellor	Review explainable student risk flags and manage follow-up tasks without initiating payment or collection actions.

Architecture and data model

The app uses a modular monolith because it keeps an MVP deployable as one service while allowing business rules to be tested independently from database access.

Request path	Responsibility
Browser UI	Next.js App Router pages and reusable React components provide public, parent, accounts, and counsellor screens.
Route handlers	Validate inputs, authenticate the request, apply role and centre access, and call a domain service.
Domain modules	Auth, ledger, payment, parent, refund, reminder, report, and risk logic. Pure calculation functions are separated from database workflows.
PostgreSQL	Prisma models hold identity, fee schedules, instalments, payments, allocations, receipts, and operational work. Neon is the expected host.

Architecture at a glance

Client	Next.js route handler	Domain service	PostgreSQL
Parent or staff UI	Session, role, and schema checks	Fee, payment, refund, reminder, report, or risk logic	Prisma data and transaction records

Main data groups

Group	Entities and purpose
Identity and access	User, Session, Guardian, Consent. Staff use roles and centre assignments; parent APIs resolve the guardian from the signed-in session.
Academic structure	Centre, Batch, Student. A student belongs to a batch; a guardian is linked to student records.
Fees	FeePlan, Enrollment, Instalment. The institute defines the plan, registration amount, discount, schedule, and due dates.
Money and evidence	Payment, Allocation, Receipt, PaymentOrder, and processed webhook records. An allocation links a received amount to an instalment.
Operations	RefundPolicy, RefundCase, Reminder, RiskFlag, CounsellorTask, attendance and test records, and AuditLog.

Financial representation

Money is stored as integer paise in BIGINT fields. A student's paid and outstanding balances are derived from instalment values and allocations in the ledger logic. The schema also stores mutable payment status and does not implement a complete double-entry journal or reversal ledger; this is a known current-state limit.

Main product flows

1. Parent access and payment

The parent enters a phone number and requests a six-digit OTP. The server stores a hash and expiry, rate-limits requests, and limits verification attempts. A verified guardian receives a session cookie; parent APIs use that session to retrieve linked students, dues, instalments, and receipts. The parent starts checkout for an institute-assigned instalment. Mock or Razorpay checkout verification records payment, allocates it to instalments, and produces a receipt when the configured settlement flow completes.

The profile screen lets the parent update their name and avatar, inspect linked student names and IDs, and choose English or Hindi for fee reminder messages. The child switcher filters the parent fee overview and receipts when multiple children are linked. The language preference changes reminder messages, not the website interface.

2. Accounts and reconciliation

Accounts staff sign in with email and password; passwords are checked against bcrypt hashes. Role and centre scope restrict data access. Staff review dues, match unmatched transfers to students or reject them with a reason, inspect refund requests, and download CSV receipt exports for a selected date range. The CSV export is an offline handoff rather than a live Tally integration.

3. Refund review

A staff member selects an enrollment, leave date, and reason. The refund service evaluates the applicable policy and the refund math uses amount paid, registration-fee treatment, course duration, and elapsed days. The workflow tracks review and settlement status; it does not transfer funds. The public calculator is an estimate and does not replace the configured institute policy.

4. Reminder automation

Vercel Cron calls `/api/cron` once daily with a bearer secret. The job processes reminder candidates and runs daily risk evaluation. Reminder templates use each guardian's saved language preference and check payment state before delivery. The database records reminder outcomes. Delivery to real SMS or WhatsApp networks is not configured in this MVP.

Payment safety properties

Concern	Current control
Duplicate webhook delivery	Gateway event and reference records are checked for replay; payment processing uses transaction boundaries and idempotency data.
Allocation	Pure ledger math allocates amounts against instalments and can represent partial payments.
Receipt traceability	A payment receipt links back to the payment and allocated student fees.
Out-of-scope guarantees	No double-entry journal, bank settlement guarantee, or statutory compliance certification is claimed.

Automation and interface design

Risk review is rule-based automation, not a generative AI feature. It creates observable signals for a counsellor rather than making an automated decision about a student.

Risk signal model

The current default weights are 45% fee, 35% attendance, and 20% academic trend. Fee features summarize overdue instalments; attendance features use recent records and trends; academic features use test-score movement. High risk requires corroborating signals. Missing data is treated as unknown rather than bad performance. Explanations are template-generated, and counsellors can confirm or mark a flag false and manage tasks.

This choice keeps risk work inspectable and reversible. The model is deterministic, its thresholds and weights are versioned in code, and a human reviewer remains responsible for follow-up. There is no connected LLM provider, model training, or accuracy monitoring dashboard.

UI and design decisions

Decision	Reason and application
Separate parent and staff entry	Parents use phone OTP while staff use password login. The two flows have different user needs and access boundaries.
Parent fee overview	Shows each linked child, fee totals, instalments, dues, and receipts. A switcher keeps multi-child context explicit.
Message preference inside profile	Keeps the Hindi/English reminder choice next to parent account information and explains that it affects messages only.
Staff workflow pages	Dues, payment matching, refunds, and exports are task-oriented operational screens rather than a decorative analytics-only dashboard.
Visual system	White and slate surfaces, indigo/violet accents, clear status colors, compact cards, responsive layouts, and motion used only as supporting detail.
Trust language	Marketing content identifies mock payments, CSV exports, and rule-based flags accurately instead of implying live integrations.

States and recovery

Portal pages have loading and API error states. Parent pages explain when no linked students or receipts exist. Forms show validation and operation feedback, OTP failures can be retried within the limit, invalid or expired sessions return to login, and staff review actions require explicit confirmation by a signed-in user. Responsive layouts target phone and desktop use.

Key UI paths

Public landing page: `/` and `/#product-ledger`; parent login and sign-up: `/login` and `/signup`; parent overview and profile: `/parent/dues` and `/parent/profile`; staff workflows are grouped under `/dashboard` and `/counsellor` according to role.

Security, verification, and next work

Security controls

Area	Control or boundary
Session	Random session tokens are stored hashed; the browser cookie is HTTP-only, SameSite Lax, and Secure in production.
Passwords and OTP	Staff passwords use bcrypt. OTPs are hashed before storage, expire, and have request and attempt limits.
Authorization	Route handlers require an authenticated parent or staff user; staff roles and centre scope limit operations.
Input handling	Request schemas use Zod validation. Payment webhooks verify signatures and check event/reference idempotency.
Secrets and data	Environment files are ignored by Git; `.env.example` uses placeholders. Seed phone values and IP addresses are synthetic/non-routable.

Verification

Vitest suites cover auth, ledger math, payment flows, refunds, reminder rules, reporting, and risk calculations. In this preparation run, TypeScript typecheck passed and the isolated demo-OTP unit suite passed (4 tests). The full database-backed suite was not run because the local environment points to a Neon database that should not be mutated as a test target. Browser-based end-to-end tests are not configured.

What is deliberately cut

Live SMS/WhatsApp delivery, connected LLM calls, institute-created fee-plan administration, parent selection among institute-approved payment plans, live Tally synchronization, multi-institute billing, and production payment settlement are outside the current verified demo scope.

Next steps

First commit and push the current working tree, then deploy that revision. Configure an isolated seeded reviewer database and the restricted parent demo OTP in Vercel. Verify one parent login, one staff login, and the mock checkout on the deployed build. After the review demo, disable the fixed OTP. Later work should add provider-backed notification delivery, a proper institute fee-plan management flow, stronger ledger correction entries, browser E2E coverage, and operational monitoring.

Release and submission status

The public GitHub repository and live app URLs at the top were reachable at preparation time. The local working tree contains uncommitted changes, including the reviewer OTP configuration, profile and portal work, and visual refinements. Those changes are not yet guaranteed to exist on public `main` or in the live deployment. Commit, push, and redeploy before relying on the public links for final review.

Implementation references: `prisma/schema.prisma`; `src/modules/auth/auth-service.ts`; `src/modules/ledger/ledger-math.ts`; `src/modules/payments/services/payment-service.ts`; `src/modules/refunds/refund-service.ts`; `src/modules/reminders/services/reminder-service.ts`; `src/modules/risk/risk-math.ts`; `vercel.json`.